

Relatório de Auditoria de Segurança

ConfortoTermico

Estado final, depois da remediação

Auditoria e remediação — 1 de setembro de 2026

Escopo auditado	Repositório ConfortoTermico na íntegra: os dois processos (<code>criar_app_ict` e <code>criar_app_coletor`</code>), as 40 rotas HTTP registradas nos 7 blueprints, o módulo de autorização por área (<code>app/auth.py`</code>, 848 linhas), as 10 camadas de persistência (<code>app/database_*.py`</code>, <code>app/dados_entrada_db.py`</code>, <code>app/db_backend.py`</code>), os 5 templates Jinja, os 9 arquivos de JavaScript próprio, <code>Dockerfile`</code>, <code>compose.yaml`</code>, <code>config/servidor.json`</code>, os arquivos de ambiente (versionados e não versionados), o workflow de CI e o histórico completo do Git.</code>
Linguagem	Python 3.13
Framework	Flask, em DOIS processos separados — <code>ict` (única interface HTTP pública) e <code>coletor` (privado, dono do cliente Modbus)</code></code>
ORM / query builder	Nenhum ORM: SQL parametrizado escrito à mão sobre psycopgp, com uma fachada de conexão por schema (<code>historico`</code> , <code>dados_entrada`</code>); migrações por Alembic
Banco	PostgreSQL 17
Mecanismo de auth	SharedAuth (sessão, CSRF, limite de login, troca de senha obrigatória, cabeçalhos) + modelo PRÓPRIO de 6 perfis mapeados a 10 áreas, aplicado por <code>before_request` com negação por padrão</code>
Frontend	SPA de abas em JavaScript de módulos ES nativos (sem framework), consumindo a API JSON do ICT; Chart.js vendorizado
Deploy	Docker Compose com 4 serviços (postgres, schema, coletor, ict), waitress como servidor WSGI, Nginx com TLS no VPS Oracle

Nota metodológica

Este é o único dos quatro aplicativos com um modelo de autorização granular próprio: 6 perfis, 10 áreas e um mapa explícito de endpoint para área. Por isso a categoria 1 (isolamento) foi traduzida não como 'filtro por dono' — não há dono de dado aqui, e sim ÁREA de uso — mas como: o mapa de áreas cobre TODOS os endpoints registrados, e o dado que cada endpoint devolve cabe na área que o libera? A segunda metade dessa pergunta é a que produziu o achado central do relatório. Todos os 40 endpoints foram enumerados e conferidos contra os três conjuntos de autorização, um a um.

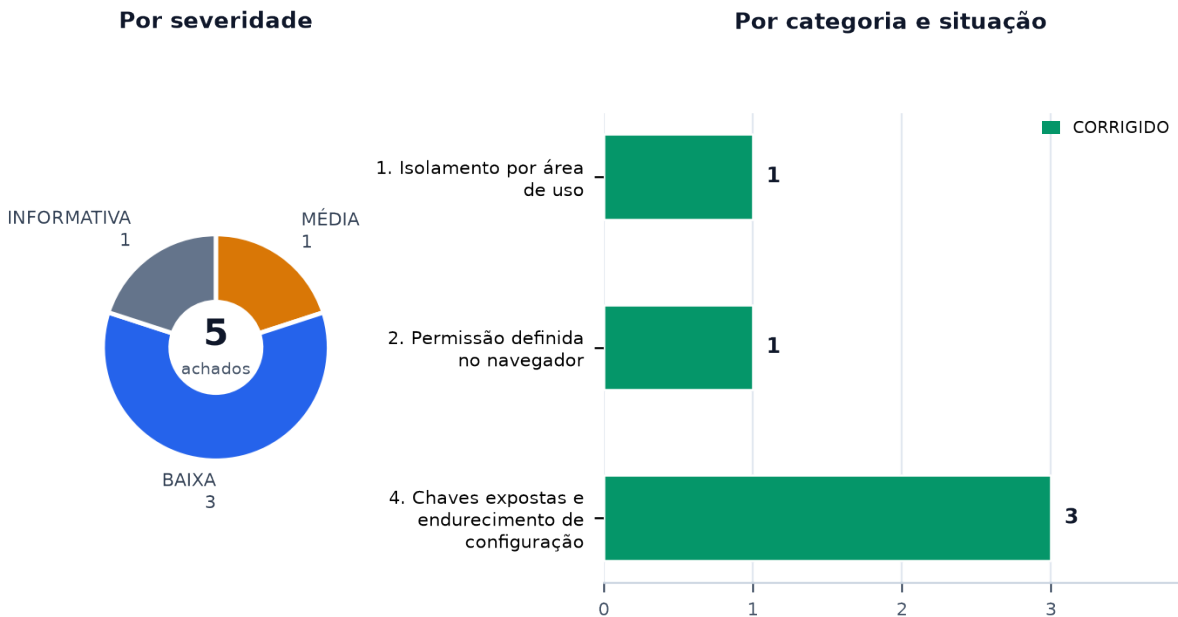
Categoria pedida	Como foi mapeada nesta stack
1. Banco sem tranca (isolamento de inquilino/dono)	Não há tenant nem dono: o isolamento é POR ÁREA DE USO, aplicado no <code>`before_request`</code> <code>`_exigir_area`</code> . Auditado em duas etapas — (a) todos os 40 endpoints registrados foram enumerados e confrontados com <code>`AREA_POR_ENDPOINT`</code> , <code>`ENDPOINTS_ISENTOS_DE_LOGIN`</code> e <code>`ENDPOINTS_ABERTOS_A_QUALQUER_PERFIL`</code> ; (b) para cada endpoint mapeado, foi verificado se a CARGA DEVOLVIDA cabe na área que o libera.
2. Permissão definida no navegador	<code>`templates/index.html`</code> recebe <code>`areas_permitidas`</code> e esconde abas por área. Cada aba foi cruzada com as rotas que ela chama, e cada rota com sua entrada no mapa de áreas. Auditado também <code>`PERFIS_EXTRA_POR_ENDPOINT`</code> , a restrição adicional por perfil sobre duas exclusões irreversíveis.
3. IDOR	Percorridos os 15 handlers que recebem <code>`zona_id`</code> , <code>`equipamento_id`</code> , <code>`execucao_id`</code> ou <code>`usuario_id`</code> . Como não há dono, a pergunta auditada foi a consistência do objeto (o equipamento pertence à zona informada?) e a autorização de área da rota.
4. Chaves expostas (hardcode)	Varredura de código, templates, <code>`compose.yaml`</code> , <code>`Dockerfile`</code> , <code>`config/servidor.json`</code> , scripts, workflow de CI, documentação, arquivos <code>`.env`</code> (inclusive os NÃO versionados) e do histórico completo do Git. Auditados também os defaults <code>`\${VAR:-valor}`</code> do Compose e o armazenamento de credencial de terceiro (SMTP) no banco.
5. Inputs sem tratamento (XSS)	Frontend: varredura dos 9 arquivos de JavaScript próprio por <code>`innerHTML`</code> , <code>`outerHTML`</code> , <code>`insertAdjacentHTML`</code> , <code>`eval`</code> , <code>`new Function`</code> , <code>`document.write`</code> e atribuição a <code>`href`</code> . Backend: 5 templates buscados por <code>` safe`</code> , <code>`autoescape`</code> e <code>`Markup`</code> ; e o corpo do e-mail de alerta, que é o único texto montado em Python.

1. Resumo executivo

Terceira leitura do mesmo relatório, depois da remediação de 1-2 de setembro de 2026. Os cinco achados desta rodada estão corrigidos e verificados por teste. O central (CT-01, o único de severidade média) fechou em 01/09: ``GET /api/zonas`` deixou de entregar a fiação Modbus a quem não tem a área ``cadastro``. Em 02/09 fechou também a lacuna de MÉTODO que permitiu o achado central passar despercebido (issue 2): a suíte agora percorre TODOS os endpoints GET do mapa com os seis perfis, afirmando sobre as chaves devolvidas — não só sobre o status HTTP —, derivando a lista do próprio ``AREA_POR_ENDPOINT`` para que um endpoint novo entre sozinho. Dos quatro achados restantes: a aplicação agora recusa subir com host público e cookie de sessão sem ``Secure`` (CT-02), e o nome do arquivo de ambiente de produção convergiu para ``env.vps`` como nos três irmãos; a senha do servidor SMTP saiu da tabela ``configuracoes`` e passou a vir exclusivamente de segredo do Compose ou variável de ambiente, com migração removendo o valor já persistido (CT-03); o token interno recusa gerar-se sozinho fora de desenvolvimento, mesma trava que a chave de sessão já tinha (CT-04); e a área ``dashboard`` está documentada como o privilégio universal que é, com teste fixando as duas metades da garantia — todo perfil a tem, nenhum endpoint de escrita pode usá-la (CT-05). O restante do quadro é o que a auditoria já descrevia e não mudou: o ``before_request`` ``_exigir_area`` nega por padrão, os 40 endpoints registrados estão cobertos pelo mapa sem exceção, a separação entre o processo do ICT e o do coletor é real, o SQL é parametrizado nas dez camadas de persistência e não há XSS. A suíte de qualidade ficou verde: Ruff sem apontamento e 368 testes.



Achados por severidade e por categoria



2. Estado da remediação

A remediação de 01/09 (CT-01) foi feita em árvore de trabalho, sem commit. A de 02/09 (P1 restante, CT-02 a CT-05) segue para commit, PR e publicação nesta mesma rodada — a forma de cada correção é o que vale registrar.

- O teste genérico (issue 2) deriva a lista de endpoints do PRÓPRIO ``AREA_POR_ENDPOINT``, não de uma lista escrita à mão: um endpoint novo mapeado fora de ``cadastro`` entra na varredura sozinho. Verificado sabotando deliberadamente ``comum.listar_zonas`` (voltando a pedir ``com_fiacao=True`` incondicionalmente) e confirmando que a suíte aponta exatamente a regressão, antes de desfazer a sabotagem.
- CT-02 e CT-04 seguem o MESMO padrão que o projeto já usava para ``CONFORTO_DEBUG`/`CONFORTO_TESTING`/`CONFORTO_SECRET_KEY``: uma função ``_validar_*``/uma trava de ``_ambiente_permite gerar_chave``, chamada no boot, com mensagem que nomeia a variável que falta. Nenhum mecanismo novo — extensão de um já revisado.
- CT-03 removeu a CHAVE do dicionário de configurações padrão, e não só o valor: isso faz a sanitização descartar sozinha qualquer ``smtpSenha`` que ainda apareça (cliente antigo, linha remanescente no banco), sem precisar de tratamento especial por chamador. A migração que apaga o valor já persistido foi verificada rodando ``alembic upgrade head`` contra um PostgreSQL real, não só lida.
- A senha SMTP passou a ser uma variável de ambiente OPCIONAL (``SMTP_PASS``), não um segredo obrigatório do Compose: SMTP é recurso opt-in, e um segredo de arquivo obrigatório teria quebrado toda instalação que hoje opera em modo simulado sem provisionar nada.

Achado	Situação	O que mudou, e como foi verificado
CT-01 MÉDIA	CORRIGIDO	<code>`GET /api/zonas`</code> entrega a fiação Modbus a todos os 6 perfis, sob a área <code>`dashboard`</code> <code>`listar_zonas`</code> ganhou o parâmetro <code>`com_fiacao`</code>, com padrão FECHADO, e projeta as colunas por <code>`COLUNAS_EQUIPAMENTO_SEM_FIACAO`</code> quando ele é falso (<code>app/database_zonas.py:219</code> e <code>:248</code>). A rota resolve o parâmetro pela área do perfil: <code>`db.listar_zonas(com_fiacao=auth.area_permitida(perfil, 'cadastro'))`</code> (<code>app/rotas_comuns.py:94-113</code>). Técnico e administrador continuam recebendo a fiação; os outros quatro perfis param de receber. Nenhum JavaScript mudou. Verificação: Sete testes novos em <code>tests/test_autorizacao_por_area.py:201</code> e <code>:222</code> — os seis perfis parametrizados, cada um afirmando que alcança a rota E se deve ou não receber a fiação, mais a invariante de que a lista de colunas públicas não pode interceptar os campos Modbus, que pega quem acrescentar <code>`host`</code> a ela no futuro.
CT-02 BAIXA	CORRIGIDO	Cookie de sessão sem <code>`Secure`</code> por padrão, sem HSTS e com o mesmo nome de arquivo em local e produção <code>`app_factory._validar_transporte(config, cookie_seguro)`</code> recusa a inicialização com <code>`RuntimeError`</code> quando <code>`CONFORTO_COOKIE_SEGURO`</code> está desligado e o host não é de loopback, mesmo padrão de <code>`_validar_debug`/`_validar_testing`</code> (<code>app/app_factory.py</code>, chamada em <code>`criar_app_ict`</code> logo onde a variável é lida). Separadamente, a divergência de nome do arquivo de ambiente foi resolvida: <code>`_manutencao/vps/deploy.sh`</code> passou a apontar <code>`ENVF=.env.vps`</code> para <code>`conforto-termico`</code> (era <code>`_env.docker`</code>), e <code>`_env.vps.example`</code> deixou de descrever a divergência como pendência. Verificação: <code>tests/test_runtime_security.py:</code> <code>test_transporte_recusa_host_publico_com_cookie_inseguro,</code> <code>test_transporte_aceita_host_publico_com_cookie_seguro,</code> <code>test_transporte_aceita_loopback_com_cookie_inseguro.</code>

Achado	Situação	O que mudou, e como foi verificado
CT-03 BAIXA	CORRIGIDO	Senha do servidor SMTP guardada em texto claro no banco `smtpSenha` deixou de ter chave em `CONFIGURACOES_PADRAO` (app/database_configuracoes.py): a sanitização descarta qualquer valor recebido de um cliente antigo, e o bloco "somente-escrita" que existia em `salvar_configuracoes` foi removido por ficar sem objeto. A migração `20260902_0001_remove_smtp_senha` apaga a linha já persistida. A senha passou a vir exclusivamente de `models._resolver_senha_smtp` (segredo do Compose via `SMTP_PASS_FILE`, ou a variável direta `SMTP_PASS`, já documentada como fallback deste app); `notificacoes.smtp_config_atual` não inclui mais "senha", deixando `Email.enviar` cair no resolvidor. `_configuracoes_publicas` passou a derivar `smtpSenhaConfigurada` do segredo, não do banco — a tela continua sabendo SE há senha, nunca qual é. O campo de senha saiu da tela (aba Configurações); host, porta e usuário continuam editáveis, porque não são segredo. Verificação: tests/test_smtp_senha_fora_do_banco.py (9 testes nova) cobre a ausência da chave em CONFIGURACOES_PADRAO, o descarte pela sanitização (valor recebido e valor remanescente da base), a ausência de "senha" em smtp_config_atual, a resolução por variável e por arquivo (com precedência do arquivo), o uso pelo Email.enviar de verdade e o retrato correto em _configuracoes_publicas. A migração foi confirmada rodando `alembic upgrade head` contra um PostgreSQL real: chega a head em 20260902_0001_remove_smtp_senha e a tabela `configuracoes` não tem mais linha `smtpSenha`.
CT-04 BAIXA	CORRIGIDO	Token interno é gerado e persistido em silêncio quando não está configurado `obter_ou_criar_token_interno` (app/auth.py) ganhou a mesma trava de `obter_chave_secreta`: fora de desenvolvimento/teste (`_ambiente_permite gerar_chave`), a ausência de `CONFORTO_INTERNO_TOKEN`/_FILE` levanta `RuntimeError` nomeando as duas variáveis e explicando a consequência (403 na API interna) em vez de gerar e persistir um token em silêncio. Verificação: tests/test_auth_required.py: test_producao_sem_token_recusa_subir (RuntimeError fora de desenvolvimento, nada gravado em disco) e test_token_interno_gerado_persiste_em_desenvolvimento (geração e persistência continuam funcionando com CONFORTO_DEVELOPMENT=1, mesmo formato do teste já existente para a chave de sessão).
CT-05 INFORMATIVA	CORRIGIDO	A área `dashboard` é privilégio universal implícito, sem teste que a fixe `AREAS_POR_PERFIL` e `AREA_POR_ENDPOINT` (app/auth.py) ganharam comentário explícito registrando que "dashboard" equivale a "qualquer conta autenticada", e que por isso só pode ser exigida por endpoint de LEITURA. A propriedade deixou de depender só do comentário: um teste novo verifica as duas metades. Verificação: tests/test_autorizacao_por_area.py: test_area_dashboard_e_universal_e_so_de_leitura confirma que todo perfil em AREAS_POR_PERFIL tem 'dashboard', e que nenhum endpoint mapeado nela aceita método de escrita (POST/PUT/PATCH/DELETE) — falha se um perfil novo nascer sem a área ou se uma rota de escrita for mapeada nela no futuro.

3. Pontos fortes e pontos fracos

3.1 O que está protegido (com evidência)

Controle verificado	Evidência no código
O controle de área NEGA o que não está mapeado <code>`_exigir_area`</code> recusa o endpoint sem entrada em <code>`AREA_POR_ENDPOINT`</code> , em vez de liberá-lo. Até 29/08/2026 era o contrário, e o preço foram seis leituras respondendo a qualquer perfil autenticado enquanto o template escondia a aba. Com a negação por padrão, esquecer o mapeamento vira uma rota que não funciona — visível na hora.	app/auth.py:526-548
Os 40 endpoints registrados estão TODOS cobertos Enumeração completa dos 7 blueprints confrontada com os três conjuntos: 33 em <code>`AREA_POR_ENDPOINT`</code> , 4 em <code>`ENDPOINTS_ISENTOS_DE_LOGIN`</code> , 5 em <code>`ENDPOINTS_ABERTOS_A_QUALQUER_PERFIL`</code> (cada um com o motivo escrito). Nenhum endpoint fora dos três.	app/auth.py:309-424
Restrição adicional por perfil sobre as exclusões irreversíveis <code>`PERFIS_EXTRA_POR_ENDPOINT`</code> limita <code>`excluir_medicoes`</code> e <code>`apagar_historico`</code> a técnico e administrador, mesmo dentro da área <code>`dados_entrada`</code> que o analista também tem. É autorização em duas camadas para a ação que apaga séries inteiras.	app/auth.py:421-425 e 137-139
Confirmação de exclusão validada NO SERVIDOR As duas rotas que zeram a série temporal exigem a palavra <code>`APAGAR`</code> no corpo da requisição, conferida por uma função compartilhada — não só por um <code>`confirm()`</code> do navegador, que uma chamada direta com sessão válida ignoraria.	app/app_factory.py:59-70
Coletor privado de verdade: sem porta publicada e com token em toda a API O serviço <code>`coletor`</code> declara <code>`expose`</code> e não <code>`ports`</code> , então não é alcançável de fora da rede do Compose. Sobre isso, um <code>`before_request`</code> confere <code>`X-Interno-Token`</code> em TODA rota exceto o health, com <code>`hmac.compare_digest`</code> — comparação em tempo constante.	compose.yaml:105-106 e app/coletor/rotas.py:57-72
Fábricas separadas de verdade <code>`criar_app_ict`</code> nunca importa <code>`coletor.estado`</code> , <code>`modbus_client`</code> nem <code>`ZonaService`</code> : a travessia para o hardware passa obrigatoriamente pela API interna. O import do blueprint do coletor é local à fábrica dele.	app/app_factory.py:293-297 e 479-489
SQL parametrizado em todas as dez camadas de persistência Nenhuma consulta interpola valor vindo do usuário. Onde há f-string, o trecho interpolado é sempre um literal escolhido pelo próprio código (um <code>`WHERE`</code> fixo) ou uma lista de marcadores <code>`?`</code> gerada a partir do TAMANHO da lista de parâmetros — nunca do conteúdo. Os nomes de coluna em <code>`dados_entrada_db`</code> vêm de <code>`_COLUNAS_MEDICAO`</code> , constante do módulo.	app/database_leituras.py:220-296 e app/dados_entrada_db.py:493-535
Senha SMTP nunca sai do servidor pela API <code>`_configuracoes_publicas`</code> zera <code>`smtpSenha`</code> e devolve só <code>`smtpSenhaConfigurada`</code> (booleano). E o campo é somente-escrita: enviar vazio significa 'não mudei', não 'apague' — sem isso, marcar qualquer checkbox apagaria a senha em silêncio.	app/ict/administracao.py:35-41 e app/database_configuracoes.py:238-239
E-mail de alerta é texto puro, com destinatário validado O corpo é <code>`MIMEText`</code> sem HTML, então não há superfície de XSS. O destinatário passa por <code>`_email_valido`</code> dentro da própria classe, antes de montar a mensagem — defesa em profundidade contra injeção de cabeçalho por quebra de linha, mesmo com a validação já feita ao salvar a configuração.	app/models.py:219-236

Controle verificado	Evidência no código
Zero ` safe` e nenhum `innerHTML` com dado do servidor Varredura dos 5 templates por ` safe`, `autoescape` e `Markup`: nada. Nos 9 arquivos de JavaScript próprio, os `innerHTML` encontrados são todos `innerHTML = ""` para limpar container; o único que recebe conteúdo (`badge.innerHTML = svgIcône`) é alimentado por `ICONE_VENTILADOR`/`ICONE_NEBULIZADOR`/`ICONE_SENSOR`, constantes literais declaradas em `app.js`.	app/static/js/features/dashboard-zonas.js:306 e app/static/js/app.js:68-80
Sessão derrubada de imediato por desativação e por troca de senha O hook que carrega `g.usuario` limpa a sessão na mesma requisição quando a conta sumiu ou foi desativada, e quando a marca da senha não corresponde mais ao hash guardado. Não espera a sessão expirar.	app/auth.py:484-510
`ProxyFix` E a configuração de `trusted_proxy` do waitress Este é o único dos quatro que usa waitress, que apaga os cabeçalhos `X-Forwarded-*` por padrão — o `ProxyFix` sozinho recebia um environ já limpo e não fazia efeito nenhum. Os dois lados agora são configurados juntos, atrás da mesma variável de ambiente, e o comentário registra por que copiar o padrão do irmão não bastava.	app/app_factory.py:333-336 e 502-546
`CONFORTO_TESTING` e `CONFORTO_DEBUG` travados fora de desenvolvimento Ligar `CONFORTO_TESTING` desliga o limitador inteiro e permite subir com chave de sessão gerada. As duas flags agora exigem `CONFORTO_DEVELOPMENT=1` e host de loopback, ou a aplicação não sobe — antes, a única garantia era a variável não estar em nenhum Compose.	app/app_factory.py:191-216
Chave de sessão obrigatória em produção, com ADR Fora de desenvolvimento/teste, a ausência de `CONFORTO_SECRET_KEY` derruba a inicialização com mensagem que diz exatamente o que fazer. A decisão está registrada em `docs/adr/007-chave-sessao-como-segredo.md`.	app/auth.py:205-222
Consulta de auditoria com teto interno `listar_eventos_auditoria` limita a 500 registros dentro da própria função (`max(1, min(500, int(limite)))`), então o `?limite=` da rota não consegue pedir uma varredura ilimitada mesmo sem validação adicional na camada HTTP.	app/database_auditoria.py:49-50
Sem segredo no histórico do Git e nenhum `.env` com valor real no disco Diferente dos três irmãos, o `.env.docker` deste projeto não contém segredo nenhum — só nomes de banco e portas. O histórico completo não tem arquivo de segredo, e o PAT do build entra por BuildKit secret.	.env.docker e git log --all
A carga de `api/zonas` é projetada pela área do perfil A rota continua liberada pela área `dashboard`, que os seis perfis têm, mas a fiação Modbus de cada equipamento só sai para quem tem `cadastro` — a mesma área que `administracao.obter_zona` exige para uma zona só.	app/rotas_comuns.py:94-113 · tests /test_autorizacao_por_area.py:222
O padrão da consulta de zonas não traz a fiação `listar_zonas` projeta colunas e só devolve o endereçamento Modbus quando explicitamente pedido. Um consumidor futuro que esqueça o parâmetro quebra com `KeyError` em vez de vaziar em silêncio.	app/database_zonas.py:219 e :248
Host público com cookie inseguro não sobe mais `_validar_transporte` recusa a inicialização quando `CONFORTO_COOKIE_SEGURO` está desligado e a escuta não é loopback, no mesmo espírito de `_validar_debug` e `_validar_testing`. Até 01/09/2026 a única garantia era o exemplo versionado do arquivo de ambiente de produção estar certo — copiar o arquivo errado desligava `Secure` em silêncio.	app/app_factory.py:_validar_transporte · tests/test_runtime_security.py

Controle verificado	Evidência no código
Produção converge para <code>.env.vps</code>, como os três irmãos Este era o único dos quatro projetos cujo <code>deploy.sh</code> lia <code>.env.docker</code> em produção — o MESMO nome do arquivo de desenvolvimento local. <code>_manutencao/vps/deploy.sh</code> e <code>.env.vps.example</code> foram atualizados juntos; a divergência de nome, registrada como pendência conhecida, está fechada.	<code>_manutencao/vps/deploy.sh</code> · <code>.env.vps.example</code>
Senha do servidor SMTP nunca mais toca o banco <code>smtpSenha</code> não tem mais chave em <code>CONFIGURACOES_PADRAO</code> : a sanitização descarta qualquer valor recebido, e uma migração Alembic remove o que já estava persistido. A senha agora vem exclusivamente de <code>SMTP_PASS</code> / <code>SMTP_PASS_FILE</code> (<code>models._resolver_senha_smtp</code>); a tela mostra apenas se HÁ senha configurada, nunca o valor.	<code>app/database_configuracoes.py</code> · <code>app/models.py:_resolver_senha_smtp</code> · <code>migrations/versions/20260902_0001_remove_smtp_senha.py</code>
Token interno não gera mais sozinho fora de desenvolvimento <code>obter_ou_criar_token_interno</code> agora levanta <code>RuntimeError</code> nomeando a variável que falta, mesma trava que <code>obter_chave_secreta</code> já tinha. Até 01/09/2026 a ausência de configuração virava uma indisponibilidade sem causa aparente (a aba Operação parando com 403 na chamada interna) em vez de um erro de inicialização.	<code>app/auth.py:obter_ou_criar_token_interno</code> · <code>tests/test_auth_required.py</code>
"dashboard" documentada como o privilégio universal que é, e fixada em teste A área continua sem renomear, mas agora tem o comentário que faltava no ponto de uso ("equivale a: qualquer conta autenticada") e um teste que falha se um perfil novo nascer sem ela ou se um endpoint de escrita for mapeado nela — a condição estrutural que permitiu CT-01 acontecer sem que ninguém notasse.	<code>app/auth.py:AREAS_POR_PERFIL</code> · <code>tests/test_autorizacao_por_area.py:test_area_dashboard_e_universal_e_so_de_leitura</code>

3.2 Os riscos centrais

- O mapa de áreas é completo, mas em um ponto ele autoriza a ROTA sem olhar a CARGA: `GET /api/zonas` exige a área `dashboard` — que os 6 perfis têm — e devolve a fiação Modbus completa de todos os equipamentos, que é exatamente o dado que a área `cadastro` existe para restringir. Um operador não vê a aba de cadastro e lê o conteúdo dela por outra porta. FECHADO em 01/09: a rota passou a projetar a carga pela área do perfil, e sete testes fixam o comportamento nos seis perfis.
- As proteções de transporte dependem de variável de ambiente com padrão inseguro: `CONFORTO_COOKIE_SEGURO` é `false` quando ausente, a aplicação não emite HSTS e não redireciona HTTP→HTTPS. Agrava-se por uma divergência conhecida e documentada: o VPS lê `.env.docker`, o MESMO nome do arquivo local — cujo exemplo versionado traz `CONFORTO_COOKIE_SEGURO=0`. FECHADO em 02/09: subir com host público e a variável desligada agora recusa a inicialização (`_validar_transporte`), e a produção passou a ler `.env.vps` — o mesmo nome dos três irmãos, não mais `.env.docker`.
- A senha do servidor SMTP é guardada em texto claro numa linha da tabela `configuracoes`. A API nunca a devolve, o que resolve o vazamento pela tela — mas não o vazamento por um dump do banco, que é justamente o artefato que o BackupRestore gera e cataloga todo dia. FECHADO em 02/09: a senha saiu da tabela `configuracoes` por completo. Uma migração Alembic remove o valor já persistido; a aplicação passa a resolver a senha exclusivamente por segredo do Compose ou variável de ambiente, nunca pelo banco — um dump novo não contém mais o valor.
- Dois segredos são gerados e persistidos em silêncio quando não estão configurados (`instance/secret_key.txt` e `instance/interno_token.txt`). O caminho da chave de sessão está travado atrás de `CONFORTO_DEVELOPMENT`; o do token interno não tem trava nenhuma. FECHADO em 02/09: `obter_ou_criar_token_interno` agora exige a mesma trava de `obter_chave_secreta` (`_ambiente_permite gerar_chave`) e recusa gerar sozinho fora de desenvolvimento, com `RuntimeError` nomeando a variável que falta.

3.3 Categorias que não se aplicam a esta stack

- **1. Banco sem tranca, na acepção de inquilino/dono** — Não há multi-tenant nem propriedade de registro: zonas, equipamentos, leituras e configurações pertencem à instalação, não a um usuário. O que existe — e foi auditado no lugar — é o isolamento POR ÁREA DE USO entre os 6 perfis. Nenhuma tabela tem coluna de dono em nenhuma das 4 revisões Alembic.
- **3. IDOR, na acepção de objeto de outro dono** — Pela mesma razão. Os 15 handlers que recebem ID foram percorridos assim mesmo: os que combinam dois identificadores conferem a consistência entre eles — ``atualizar Equipamento``, ``excluir Equipamento`` e ``testar_conexao Equipamento`` recusam com 404 quando ``Equipamento[zona_id] != zona_id``, tanto no ICT quanto no coletor. Não foi encontrado handler que altere objeto sem essa checagem.
- **5. XSS por markdown ou sanitização ausente** — O projeto não renderiza markdown, não tem editor de conteúdo rico e não instala biblioteca de sanitização — porque não precisa: nenhum ponto da aplicação injeta HTML montado a partir de entrada do usuário. O único HTML produzido em Python é o SVG de ícone do SharedAuth, cujo traçado vem de um dicionário fechado.

4. Achados detalhados

Um bloco por categoria. Todo achado abaixo foi conferido no código real: arquivo, linha e trecho vêm da árvore auditada, não de inferência. O diagnóstico está preservado exatamente como foi escrito na auditoria — inclusive nos achados já corrigidos, cujo trecho de código citado não existe mais na árvore. O que mudou vem depois dele, no bloco de correção.

1. Isolamento por área de uso

Severidade	Arquivo:linha	Descrição
MÉDIA	app/rotas_comuns.py:94 -96 + app/auth.py:368 + app/database_zonas.py:209-245	[CT-01] `GET /api/zonas` entrega a fiação Modbus a todos os 6 perfis, sob a área `dashboard` O endpoint `comum.listar_zonas` está mapeado para a área `dashboard`, que os 6 perfis possuem — inclusive `operador`, cujo conjunto é apenas `{dashboard, operacao}`. A carga devolvida vem de `db.listar_zonas()`, que anexa a cada zona a lista completa de equipamentos obtida por `SELECT * FROM equipamentos`. Essa tabela tem, além de nome e tipo, as colunas `host`, `porta`, `porta_serial`, `baud_rate`, `unidade_id`, `tipo_registrador` e `endereço_registrador`. Por que é explorável: Essa é exatamente a informação que a área `cadastro` existe para restringir. O próprio mapa registra a intenção: o bloco de `administracao.*` está comentado como 'Cadastro (zonas/equipamentos, fiação Modbus)' e `administracao.obter_zona` — que devolve UMA zona com os mesmos campos — exige `cadastro`, disponível só para técnico e administrador. Ou seja: a rota de uma zona está fechada, e a rota de TODAS as zonas está aberta, com o mesmo conteúdo. Um perfil `operador`, `gestor`, `veterinário` ou `analista` autenticado obtém o inventário completo com um `GET /api/zonas`, sem nenhum privilégio adicional e sem que a interface ofereça o caminho. Impacto: Divulgação do mapa de endereçamento da rede industrial: IP e porta de cada CLP/sensor, ou a porta serial e o baud rate, mais o `unidade_id` e o endereço de registrador de cada ponto. É o conjunto exato de que alguém precisaria para falar com o equipamento diretamente, sem passar pela aplicação, se alcançar a rede. Não é escrita nem escalonamento — é a primeira metade de um ataque contra o processo físico. Condição de explorabilidade: Basta uma sessão válida de qualquer perfil. Não requer privilégio, configuração insegura nem feature flag. A mesma carga NÃO vaza por `comum.status_operacao` nem por `comum.historicos_recents_zonas`: `obter_estado_operacional_zonas` projeta campo a campo e não inclui equipamentos (verificado em app/database_zonas.py:273-337). <pre># app/auth.py:368 "comum.listar_zonas": "dashboard", # app/database_zonas.py:222 linhas_equipamentos = conn.execute("SELECT * FROM equipamentos ORDER BY zona_id, tipo, id").fetchall()</pre> CORRIGIDO — o que mudou: `listar_zonas` ganhou o parâmetro `com_fiacao`, com padrão FECHADO, e projeta as colunas por `COLUNAS_EQUIPAMENTO_SEM_FIACAO` quando ele é falso (app/database_zonas.py:219 e :248). A rota resolve o parâmetro pela área do perfil: `db.listar_zonas(com_fiacao=auth.area_permitida(perfil, 'cadastro'))` (app/rotas_comuns.py:94-113). Técnico e administrador continuam recebendo a fiação; os outros quatro perfis param de receber. Nenhum JavaScript mudou. Verificação: Sete testes novos em tests/test_autorizacao_por_area.py:201 e :222 — os seis perfis

Severidade	Arquivo:linha	Descrição
		parametrizados, cada um afirmando que alcança a rota E se deve ou não receber a fiação, mais a invariante de que a lista de colunas públicas não pode interceptar os campos Modbus, que pega quem acrescentar `host` a ela no futuro.

2. Permissão definida no navegador

Severidade	Arquivo:linha	Descrição
INFORMATIVA CORRIGIDO	app/auth.py:90-111 e 366-369	[CT-05] A área `dashboard` é privilégio universal implícito, sem teste que a fixe `dashboard` é a única área presente em todos os 6 perfis, e o comentário registra o motivo: 'a aba Dashboard nunca tem uma ação de escrita — é só exibição'. Três endpoints estão mapeados nela (`comum.index` está fora, entre os abertos a qualquer perfil). Nada no código impede que um endpoint com carga sensível seja acrescentado a essa área no futuro, e nenhum teste afirma que `dashboard` é somente-leitura. Por que é explorável: Não é uma falha por si: é a condição estrutural que permitiu CT-01 acontecer sem que ninguém notasse. Mapear um endpoint para `dashboard` é, na prática, torná-lo público para toda conta autenticada — e a redação do mapa não deixa isso evidente no ponto da escrita. Impacto: Nenhum imediato além de CT-01. Registrado porque a correção de CT-01 é pontual e esta é a causa que a faria voltar. Condição de explorabilidade: Não é explorável: é uma propriedade do desenho. "comum.listar_zonas": "dashboard", "comum.historicos_recentes_zonas": "dashboard", CORRIGIDO — o que mudou: `AREAS_POR_PERFIL` e `AREA_POR_ENDPOINT` (app/auth.py) ganharam comentário explícito registrando que "dashboard" equivale a "qualquer conta autenticada", e que por isso só pode ser exigida por endpoint de LEITURA. A propriedade deixou de depender só do comentário: um teste novo verifica as duas metades. Verificação: tests/test_autorizacao_por_area.py:test_area_dashboard_e_universal_e_so_de_leitura confirma que todo perfil em AREAS_POR_PERFIL tem 'dashboard', e que nenhum endpoint mapeado nela aceita método de escrita (POST/PUT/PATCH/DELETE) — falha se um perfil novo nascer sem a área ou se uma rota de escrita for mapeada nela no futuro.

4. Chaves expostas e endurecimento de configuração

Severidade	Arquivo:linha	Descrição
BAIXA CORRIGIDO	app/app_factory.py:305 + .env.docker.example:9 + .env.vps.example:14-20	[CT-02] Cookie de sessão sem `Secure` por padrão, sem HSTS e com o mesmo nome de arquivo em local e produção `CONFORTO_COOKIE_SEGURO` é lido com padrão `False` e é o único interruptor que liga `Secure` nos cookies. A aplicação não emite `Strict-Transport-Security` e não redireciona HTTP→HTTPS. Somando-se a isso, este é o único dos quatro projetos cujo `deploy.sh` lê `.env.docker` em produção — o MESMO nome do arquivo de desenvolvimento —, e o exemplo versionado desse arquivo traz `CONFORTO_COOKIE_SEGURO=0`. Por que é explorável: A coincidência de nomes transforma um erro de cópia num rebaixamento silencioso de segurança: copiar o `.env.docker` local para o servidor — operação que parece correta, porque o nome é o

Severidade	Arquivo:linha	Descrição
		mesmo — desliga o `Secure` do cookie de sessão e o `ProxyFix`. Nada falha, nada avisa. Sem HSTS emitido pela aplicação, a garantia de transporte fica inteiramente na diretiva do Nginx, que vive fora deste repositório. Impacto: Numa implantação com a variável ausente ou em `0`: o cookie de sessão passa a ser enviado também em requisições HTTP, interceptável antes do redirecionamento; e sem `CONFORTO_TRUST_PROXY_HEADERS` o limitador volta a chavear pelo IP do gateway do Docker — um balde único para todos os clientes, que é o defeito que este projeto já teve e corrigiu. Condição de explorabilidade: A produção atual está correta e foi conferida no VPS em 2026-08-21; `.env.vps.example` documenta `CONFORTO_COOKIE_SEGURO=1` e `CONFORTO_TRUST_PROXY_HEADERS=true`, e registra a divergência de nome como pendência conhecida. O achado é sobre a ausência de trava que impeça a configuração insegura de subir. <pre>https_obrigatorio=_ler_bool_env("CONFORTO_COOKIE_SEGURO", False),</pre> CORRIGIDO — o que mudou: `app_factory._validar_transporte(config, cookie_seguro)` recusa a inicialização com `RuntimeError` quando `CONFORTO_COOKIE_SEGURO` está desligado e o host não é de loopback, mesmo padrão de `_validar_debug`/_`validar_testing` (app/app_factory.py, chamada em `criar_app_ict` logo onde a variável é lida). Separadamente, a divergência de nome do arquivo de ambiente foi resolvida: `_manutencao/vps/deploy.sh` passou a apontar `ENVF=.env.vps` para `conforto-termico` (era `.env.docker`), e `.env.vps.example` deixou de descrever a divergência como pendência. Verificação: tests/test_runtime_security.py: test_transporte_recusa_host_publico_com_cookie_inseguro, test_transporte_aceita_host_publico_com_cookie_seguro, test_transporte_aceita_loopback_com_cookie_inseguro.
Severidade	Arquivo:linha	Descrição
BAIXA CORRIGIDO	app/database_configura coes.py:41 e 232-245	[CT-03] Senha do servidor SMTP guardada em texto claro no banco A configuração `smtpSenha` é persistida como valor de texto na tabela `configuracoes`, sem cifra e sem derivação. A API a trata corretamente como campo somente-escrita — `_configuracoes_publicas` a substitui por uma string vazia e expõe apenas o booleano `smtpSenhaConfigurada` — mas o valor em repouso é a senha em claro. Por que é explorável: O tratamento somente-escrita fecha o vazamento pela TELA, e é o certo. O que ele não cobre é o vazamento pelo DADO EM REPOUSO — e este projeto tem, por desenho, um artefato que replica exatamente esse dado: o BackupRestore gera, cataloga e verifica um dump do banco todo dia. Quem tiver acesso a um dump, a um backup restaurado, a uma réplica de desenvolvimento ou à porta 5002 do Postgres lê a senha do servidor SMTP sem passar por nenhuma tela. Impacto: Comprometimento de uma credencial de terceiro (a conta de e-mail usada para enviar alerta), que costuma valer para muito mais do que enviar alerta: envio de mensagem em nome da organização, e frequentemente leitura da caixa. É credencial de outro sistema, não desta aplicação. Condição de explorabilidade: Só existe se o SMTP tiver sido configurado pela tela. Com os campos vazios, o sistema opera em modo simulado e a coluna guarda string vazia. O fallback por variável de ambiente (`SMTP_PASS`) não passa pelo banco. <pre>"smtpHost": "", "smtpPorta": 587, "smtpUsuario": "", "smtpSenha": "",</pre>

Severidade	Arquivo:linha	Descrição
		CORRIGIDO — o que mudou: `smtpSenha` deixou de ter chave em `CONFIGURACOES_PADRAO` (app/database_configuracoes.py): a sanitização descarta qualquer valor recebido de um cliente antigo, e o bloco "somente-escrita" que existia em `salvar_configuracoes` foi removido por ficar sem objeto. A migração `20260902_0001_remover_smtp_senha` apaga a linha já persistida. A senha passou a vir exclusivamente de `models._resolver_senha_smtp` (segredo do Compose via `SMTP_PASS_FILE`, ou a variável direta `SMTP_PASS`, já documentada como fallback deste app); `notificacoes.smtp_config_atual` não inclui mais "senha", deixando `Email.enviar` cair no resolver. `_configuracoes_publicas` passou a derivar `smtpSenhaConfigurada` do segredo, não do banco — a tela continua sabendo SE há senha, nunca qual é. O campo de senha saiu da tela (aba Configurações); host, porta e usuário continuam editáveis, porque não são segredo. Verificação: tests/test_smtp_senha_fora_do_banco.py (9 testes nova) cobre a ausência da chave em CONFIGURACOES_PADRAO, o descarte pela sanitização (valor recebido e valor remanescente da base), a ausência de "senha" em smtp_config_atual, a resolução por variável e por arquivo (com precedência do arquivo), o uso pelo Email.enviar de verdade e o retrato correto em _configuracoes_publicas. A migração foi confirmada rodando `alembic upgrade head` contra um PostgreSQL real: chega a head em 20260902_0001_remover_smtp_senha e a tabela `configuracoes` não tem mais linha `smtpSenha`.
Severidade	Arquivo:linha	Descrição
BAIXA CORRIGIDO	app/auth.py:256-296	[CT-04] Token interno é gerado e persistido em silêncio quando não está configurado `obter_ou_criar_token_interno` tenta, nesta ordem: variável de ambiente, arquivo de segredo, arquivo em `instance/interno_token.txt`. Não encontrando nenhum, gera um `secrets.token_hex(32)` e tenta persistir. Se a escrita falhar (o contêiner tem `read_only: true`), o token vale só para aquele processo. Diferente de `obter_chave_secreta`, este caminho NÃO tem a trava de `_ambiente_permite gerar_chave`. Por que é explorável: O desfecho é seguro — ICT e coletor geram tokens diferentes e a API interna passa a recusar tudo com 403 —, mas o modo de falhar é ruim: a configuração ausente vira uma indisponibilidade sem causa aparente, em vez de um erro de inicialização que nomeia a variável. O contraste está no mesmo arquivo: a chave de sessão recusa-se a ser gerada fora de desenvolvimento, e a mensagem diz o que fazer. Impacto: Baixo e não é escalonamento de privilégio: falha fechada, não aberta. O custo é de diagnóstico — a aba Operação para de funcionar por inteiro e o log só mostra 403 na chamada interna. Condição de explorabilidade: Só ocorre se `CONFORTO_INTERNO_TOKEN_FILE` e `CONFORTO_INTERNO_TOKEN` estiverem ambos ausentes. O `compose.yaml` os define para os dois serviços (linhas 101 e correspondente no `ict`), então não acontece na implantação atual. <pre> novo_token = secrets.token_hex(32) try: caminho.parent.mkdir(parents=True, exist_ok=True) caminho.write_text(novo_token, encoding="utf-8") ... except OSError: pass return novo_token </pre> CORRIGIDO — o que mudou: `obter_ou_criar_token_interno`

Severidade	Arquivo:linha	Descrição
		(app/auth.py) ganhou a mesma trava de <code>`obter_chave_secreta`</code>: fora de desenvolvimento/teste (<code>`_ambiente_permite gerar_chave`</code>), a ausência de <code>`CONFORTO_INTERNO_TOKEN`/_FILE`</code> levanta <code>`RuntimeError`</code> nomeando as duas variáveis e explicando a consequência (403 na API interna) em vez de gerar e persistir um token em silêncio. Verificação: tests/test_auth_required.py: test_producao_sem_token_recusa_subir (RuntimeError fora de desenvolvimento, nada gravado em disco) e test_token_interno_gerado_persiste_em_desenvolvimento (geração e persistência continuam funcionando com CONFORTO_DEVELOPMENT=1, mesmo formato do teste já existente para a chave de sessão).

5. Recomendações priorizadas

Prio	Ação	Achados
P1 FEITA	Projetar a carga de `comum.listar_zonas` para o que a área `dashboard` justifica. Substituir o `SELECT *` por uma projeção explícita dos campos de equipamento que o dashboard de fato usa — a varredura do JavaScript mostra que ele lê apenas `tipo` (para contar ícones por zona) — e deixar `host`, `porta`, `porta_serial`, `baud_rate`, `unidade_id`, `tipo_registrador` e `endereco_registrador` exclusivamente em `administracao.obter_zona`, que já exige `cadastro`.	CT-01
P1 FEITA	Acrescentar à suíte um teste que exercite CADA endpoint do mapa com CADA um dos 6 perfis e afirme sobre os CAMPOS devolvidos, não só sobre o código HTTP. O teste atual de autorização por área confirma quem entra; falta o que sai. É esse teste que impede CT-01 de voltar por outra rota.	CT-01, CT-05
P2 FEITA	Fazer a aplicação recusar-se a subir quando o host confiável for público e `CONFORTO_COOKIE_SEGURO` estiver desligado, no mesmo espírito de `_validar_debug` e `_validar_testing`, que já existem e resolvem exatamente esse tipo de configuração perigosa em silêncio. E resolver a divergência de nome do arquivo de ambiente entre local e VPS, que já está registrada como pendência em `.env.vps.example`.	CT-02
P2 FEITA	Tirar a senha SMTP do banco: manter a credencial exclusivamente no caminho que já existe como fallback (`SMTP_PASS` por variável ou segredo do Compose), e deixar na tela apenas host, porta e usuário. Se a edição da senha pela tela for requisito, cifrá-la com uma chave que não viaje junto do dump.	CT-03
P3 FEITA	Fazer `obter_ou_criar_token_interno` levantar `RuntimeError` fora de desenvolvimento quando nenhuma das duas variáveis estiver configurada, reaproveitando `_ambiente_permite gerar_chave`, que já existe no mesmo módulo. Falhar ao subir com uma mensagem que nomeia a variável é melhor que uma aba que para de funcionar sem explicação.	CT-04
P3 FEITA	Renomear ou documentar a área `dashboard` como o privilégio universal que ela é — por exemplo, um comentário no ponto do mapa dizendo 'esta área equivale a: qualquer conta autenticada' — e fixar essa propriedade num teste que falhe se um perfil novo nascer sem ela ou se um endpoint de escrita for mapeado nela.	CT-05

6. Issues para o GitHub

Cada bloco abaixo é o texto COMPLETO de uma issue, em Markdown, pronto para copiar e colar. Achados triviais do mesmo tema foram agrupados numa issue só, para não gerar ruído no rastreador. As issues já resolvidas continuam aqui, marcadas — apagá-las deixaria o relatório sem o registro do que foi fechado —, mas não devem ser abertas no rastreador.

✓ Issue 1 — [Segurança] `GET /api/zonas` expõe a fiação Modbus a todos os perfis

```
--- ISSUE 1 ---
*** JÁ RESOLVIDA -- NÃO ABRIR NO RASTREADOR ***

Título: [Segurança] `GET /api/zonas` expõe a fiação Modbus a todos os perfis
Labels: security, severity:media, autorizacao

## Problema

O endpoint `comum.listar_zonas` está mapeado para a área `dashboard`, que os seis perfis possuem — inclusive `operador`, cujo conjunto é apenas `{dashboard, operacao}`. A carga devolvida vem de `db.listar_zonas()`, que anexa a cada zona a lista completa de equipamentos obtida por `SELECT * FROM equipamentos`.

Essa tabela tem, além de nome e tipo: `host`, `porta`, `porta_serial`, `baud_rate`, `unidade_id`, `tipo_registrador` e `endereço_registrador`.

### Por que é explorável

Essa é exatamente a informação que a área `cadastro` existe para restringir. O próprio mapa registra a intenção — o bloco de `administracao.*` está comentado como "Cadastro (zonas/equipamentos, fiação Modbus)" — e `administracao.obter_zona`, que devolve UMA zona com os mesmos campos, exige `cadastro`, disponível só para técnico e administrador.

Ou seja: a rota de uma zona está fechada e a rota de TODAS as zonas está aberta, com o mesmo conteúdo. Basta uma sessão válida de qualquer perfil:

---

GET /api/zonas
Cookie: conforto_session=<sessao de um operador>
---

Sem privilégio adicional, sem configuração insegura, sem feature flag — e sem que a interface ofereça o caminho, porque a aba de cadastro está escondida para esse perfil.

## Evidência

`app/auth.py:368`

```python
"comum.listar_zonas": "dashboard",
```

`app/auth.py:372` — a rota equivalente, para uma zona só:

```python
"administracao.obter_zona": "cadastro",
```

`app/auth.py:90-91` — quem tem `dashboard`:

```python
AREAS_POR_PERFIL: dict[str, frozenset[str]] = {
 "operador": frozenset({"dashboard", "operacao"}),
}
```

`app/database_zonas.py:222` — a carga:

```python
linhas_equipamentos = conn.execute(
 "SELECT * FROM equipamentos ORDER BY zona_id, tipo, id"
).fetchall()
```

`migrations/versions/20260803_0001_baseline.py:24` — o que `SELECT *` traz:

```sql
CREATE TABLE historico.equipamentos (
 id BIGINT ..., zona_id BIGINT ..., tipo TEXT, nome TEXT,
 modo_conexao TEXT NOT NULL, host TEXT, porta INTEGER, porta_serial TEXT,
 baud_rate INTEGER, unidade_id INTEGER NOT NULL DEFAULT 1,
 tipo_registrador TEXT NOT NULL, endereco_registrador INTEGER NOT NULL, ...)
```
```

```
## Impacto

Divulgação do mapa de endereçamento da rede industrial: IP e porta de cada
CLP/sensor (ou porta serial e baud rate), mais `unidade_id` e endereço de
registrador de cada ponto. É o conjunto exato de que alguém precisaria para
falar com o equipamento diretamente, sem passar pela aplicação, se alcançar a
rede. Não é escrita nem escalonamento – é a primeira metade de um ataque contra
o processo físico.

## Sugestão de correção

Projetar a carga para o que a área justifica. A varredura do JavaScript mostra
que o dashboard lê apenas `tipo` dos equipamentos (`equipamentosDaZona` em
`app/static/js/features/dashboard-zonas.js:528` filtra por `e.tipo`); quem lê
os demais campos é `cadastro-zonas.js`, na aba que já exige `cadastro`.

Em `app/database_zonas.py`, trocar o `SELECT *` por uma projeção e dar a
`listar_zonas` um parâmetro que decida o nível de detalhe:

```python
_COLUNAS_EQUIPAMENTO_PUBLICAS = "id, zona_id, tipo, nome, campo_medido"

def listar_zonas(*, apenas_ativas=False, com_fiacao=False):
 ...
 colunas = "" if com_fiacao else _COLUNAS_EQUIPAMENTO_PUBLICAS

`comum.listar_zonas` chama sem `com_fiacao`; `administracao.obter_zona` e
`administracao.atualizar_zona`, que já exigem `cadastro`, chamam com.

Critérios de aceite

- [] `GET /api/zonas` não devolve `host`, `porta`, `porta_serial`, `baud_rate`, `unidade_id`,
 `tipo_registrador` nem `endereço_registrador`
- [] `GET /api/zonas/<id>` (área `cadastro`) continua devolvendo todos esses campos
- [] Teste: sessão de `operador` chama `GET /api/zonas` e a resposta NÃO contém nenhuma das
 sete chaves acima
- [] Teste: sessão de `tecnico` chama `GET /api/zonas/<id>` e a resposta contém todas elas
- [] O Dashboard continua desenhando os ícones de ventilador, nebulizador e sensor por zona
 (verificado na tela)
- [] A aba Zonas continua editando a fiação normalmente (verificado na tela)
- [] `tests/test_autorizacao_por_area.py` ganha o caso novo
--- FIM ISSUE 1 ---
```

## ✓ Issue 2 — [Segurança] Teste de autorização por área deve afirmar sobre os campos devolvidos, não só sobre o status

```
--- ISSUE 2 ---
*** JÁ RESOLVIDA -- NÃO ABRIR NO RASTREADOR ***

Título: [Segurança] Teste de autorização por área deve afirmar sobre os campos devolvidos, não
só sobre o status
Labels: security, severity:media, testes, autorizacao

Problema

`tests/test_autorizacao_por_area.py` confirma QUEM entra em cada endpoint: para
cada perfil, a rota responde ou é negada conforme o mapa. O que ele não afirma é
O QUE SAI. Foi essa lacuna que deixou passar a exposição da fiação Modbus por
`GET /api/zonas` (issue 1): a rota está autorizada corretamente para a área
`dashboard` – o problema é a carga não caber nessa área.

Por que importa

A correção da issue 1 é pontual: muda uma consulta. Sem um teste que olhe o
conteúdo, a mesma classe de defeito volta na próxima rota que reaproveitar
`listar_zonas` ou qualquer outra função de leitura ampla – e volta em silêncio,
porque o teste de autorização continuaria verde.

Evidência

`app/auth.py:526-548` – o portão que o teste cobre hoje:

```python
@app.before_request
def _exigir_area() -> ResponseReturnValue | None:
    ...
    area_requerida = AREA_POR_ENDPOINT.get(endpoint)
    if area_requerida is None:
        return _negar_acesso()
    ...

O mapa cobre os 40 endpoints registrados (auditado um a um, sem furo). A lacuna
não é de cobertura de rota – é de cobertura de conteúdo.

## Impacto

Uma incoerência entre a área que autoriza e o dado que a rota entrega passa pela
```

```

súite inteira sem ser notada. É o modo de falha mais provável deste desenho,
justamente porque o mapa de áreas está correto e dá a impressão de que a
questão está resolvida.

## Sugestão de correção

Acrescentar ao teste existente uma tabela de campos PROIBIDOS por área, e
percorrer os endpoints de leitura com cada perfil afirmando sobre o corpo JSON:

```python
CAMPOS_RESTRIITOS_A_CADASTRO = {
 "host", "porta", "porta_serial", "baud_rate",
 "unidade_id", "tipo_registrador", "endereço_registrador",
}

def test_leitura_nao_vaza_campo_de_outra_area(cliente, perfil):
 # para todo endpoint GET mapeado numa área que NÃO seja `cadastro`,
 # nenhuma chave do JSON devolvido pode estar em CAMPOS_RESTRIITOS_A_CADASTRO
 ...

A varredura deve ser recursiva sobre o JSON (as zonas trazem equipamentos
aninhados) e derivar a lista de endpoints do próprio `ÁREA_POR_ENDPOINT`, para
que uma rota nova entre no teste sozinha – o mesmo princípio que faz o mapa
negar por padrão.

Critérios de aceite

- [] Existe teste que percorre os endpoints GET de `ÁREA_POR_ENDPOINT` com cada um dos 6 perfis
- [] O teste afirma sobre as CHAVES do JSON devolvido, recursivamente
- [] Um endpoint novo mapeado fora de `cadastro` entra no teste automaticamente, sem edição de
 lista
- [] O teste falha se a issue 1 for revertida
- [] A suite continua verde depois da correção da issue 1
--- FIM ISSUE 2 ---

```

### ✓ Issue 3 — [Segurança] Cookie sem Secure por padrão e mesmo nome de arquivo de ambiente em local e produção

```

--- ISSUE 3 ---
*** JÁ RESOLVIDA -- NÃO ABRIR NO RASTREADOR ***

Título: [Segurança] Cookie sem Secure por padrão e mesmo nome de arquivo de ambiente em local e
produção
Labels: security, severity:baixa, implantacao

Problema

Três coisas se somam:

1. `CONFORTO_COOKIE_SEGURO` é lido com padrão `False`, e é o único interruptor
 que liga `Secure` nos cookies de sessão;
2. a aplicação não emite `Strict-Transport-Security` e não redireciona
 HTTP→HTTPS – os dois existem apenas no Nginx do VPS;
3. este é o único dos quatro projetos cujo `deploy.sh` lê `.env.docker` em
 produção – o MESMO nome do arquivo de desenvolvimento –, e o exemplo
 versionado desse arquivo traz `CONFORTO_COOKIE_SEGURO=0`.

Por que é explorável

A coincidência de nomes transforma um erro de cópia num rebaixamento silencioso
de segurança. Copiar o `.env.docker` local para o servidor parece a operação
certa, porque o nome bate – e desliga de uma vez o `Secure` do cookie e o
`ProxyFix`. Nada falha, nada avisa.

Sem `CONFORTO_TRUST_PROXY_HEADERS`, o limitador volta a chavear pelo IP do
gateway do Docker: um balde único somando todos os clientes. É exatamente o
defeito que este projeto já teve e corrigiu, documentado em
`app/app_factory.py:309-336`.

Evidência

`app/app_factory.py:305`

```python
    https_obrigatorio=_ler_bool_env("CONFORTO_COOKIE_SEGURO", False),
    ...

`.env.docker.example:9` – a linha de base LOCAL, correta para HTTP:
...

CONFORTO_COOKIE_SEGURO=0
...

`.env.vps.example:14-20` – a divergência, já registrada como pendência:
...

# DIVERGENCIA CONHECIDA, ainda nao resolvida: os outros tres projetos leem

```

```
# `.env.vps` em producao; aqui o `deploy.sh` le `.env.docker` (o mesmo nome do
# arquivo local, com conteudo diferente).
```

Impacto

Numa implantação com a variável ausente ou em `0`: cookie de sessão enviado
também em requisições HTTP, interceptável antes do redirecionamento; e proteção
de força bruta do login sem efeito prático.

A produção atual está correta – conferida no VPS em 2026-08-21. O achado é sobre
a ausência de trava que impeça a configuração insegura de subir.

Sugestão de correção

Este projeto já tem o padrão certo para isso, em duas funções: `_validar_debug`
e `_validar_testing` recusam configuração perigosa no boot. Acrescentar a
terceira, no mesmo estilo:

```python
def _validar_transporte(config: AppConfig, cookie_seguro: bool) -> None:
    """Host publico com cookie inseguro nao sobe."""
    if cookie_seguro or config.host in HOSTS_LOOPBACK:
        return
    raise RuntimeError(
        "CONFORTO_COOKIE_SEGURO desligado com escuta fora de loopback: "
        "o cookie de sessao sairia sem Secure."
    )
```

E resolver a divergência de nome, unificando o arquivo de produção em
`.env.vps` como nos três irmãos – mexendo no `deploy.sh` e no servidor na mesma
rodada, como o próprio `.env.vps.example` já prevê.

Critérios de aceite

- [] Subir com escuta fora de loopback e `CONFORTO_COOKIE_SEGURO=0` levanta `RuntimeError`
 nomeando a variável
- [] Subir em `127.0.0.1` com a variável desligada continua funcionando (desenvolvimento local
 em HTTP)
- [] Teste cobrindo os dois casos, no estilo dos testes de `_validar_debug`
- [] O arquivo de ambiente de produção passa a se chamar `.env.vps`, com `deploy.sh` e servidor
 atualizados na mesma mudança
- [] `docs/RUNBOOK.md` registra o novo nome e a checagem de boot
--- FIM ISSUE 3 ---
```

## ✓ Issue 4 — [Segurança] Senha SMTP em texto claro no banco (e, portanto, nos backups)

```
--- ISSUE 4 ---
*** JÁ RESOLVIDA -- NÃO ABRIR NO RASTREADOR ***

Título: [Segurança] Senha SMTP em texto claro no banco (e, portanto, nos backups)
Labels: security, severity:baixa, segredos

Problema

A configuração `smtpSenha` é persistida como valor de texto na tabela
`configuracoes`, sem cifra e sem derivação.

O tratamento na API está correto e não é o problema: `_configuracoes_publicas`
substitui a senha por string vazia e expõe apenas o booleano
`smtpSenhaConfigurada`; e o campo é somente-escrita, então enviar vazio
significa "não mudei", não "apague". Isso fecha o vazamento pela tela.

Por que é explorável

O que o tratamento somente-escrita não cobre é o dado EM REPOUSO – e este
projeto tem, por desenho, um artefato que replica exatamente esse dado: o
BackupRestore gera, cataloga e verifica um dump do banco todo dia.

Quem tiver acesso a um dump, a um backup restaurado, a uma réplica de
desenvolvimento ou à porta 5002 do Postgres lê a senha do servidor SMTP sem
passar por nenhuma tela e sem explorar nenhuma falha da aplicação.

Evidência

`app/database_configuracoes.py:38-41`

```python
    "smtpHost": "",
    "smtpPorta": 587,
    "smtpUsuario": "",
    "smtpSenha": "",
```

`app/database_configuracoes.py:238-239` – o tratamento somente-escrita, que está
certo e resolve outra coisa:
```

```

```python
    if not str(configuracoes.get("smtpSenha", "")).strip():
        configuracoes.pop("smtpSenha", None)
...

`app/models.py:231` – onde o valor é consumido:
```python
 senha = str(smtp_config.get("senha") or os.environ.get("SMTP_PASS") or "")
...

Impacto

Comprometimento de uma credencial de TERCEIRO – a conta de e-mail usada para
enviar alerta –, que costuma valer para muito mais do que enviar alerta: envio
de mensagem em nome da organização e, com frequência, leitura da caixa. É
credencial de outro sistema, não desta aplicação, e por isso o estrago não fica
contido aqui.

Condição: só existe se o SMTP tiver sido configurado pela tela. Com os
campos vazios o sistema opera em modo simulado e a coluna guarda string vazia.

Sugestão de correção

O caminho mais simples e mais seguro já existe no código: `Email.enviar` aceita
`SMTP_PASS` como fallback por variável de ambiente. Tirar a senha do banco por
completo:

- na tela, manter host, porta e usuário (que não são segredo);
- a senha passa a vir exclusivamente de segredo do Compose
 (`SMTP_PASSWORD_FILE`, no mesmo padrão dos outros três segredos deste
 projeto);
- remover a chave `smtpSenha` do documento de configurações, com revisão
 Alembic que apague o valor existente.

Se a edição pela tela for requisito operacional, a alternativa é cifrar o valor
com uma chave que não viaje junto do dump – o que exige um segredo a mais e
um plano de rotação, e por isso é a segunda opção, não a primeira.

Critérios de aceite

- [] A senha SMTP não é mais gravada na tabela `configuracoes`
- [] Revisão Alembic remove o valor já existente da coluna
- [] O envio de e-mail de alerta continua funcionando com a credencial vinda de segredo do
 Compose
- [] `_configuracoes_publicas` continua devolvendo `smtpSenhaConfigurada` para a tela saber o
 estado
- [] `docs/RUNBOOK.md` registra como provisionar e rotacionar a credencial SMTP
- [] Um dump novo do banco não contém a senha (verificado com `grep` no dump)
--- FIM ISSUE 4 ---

```

## ✓ Issue 5 — [Segurança] Token interno gerado em silêncio quando não configurado

```

--- ISSUE 5 ---
*** JÁ RESOLVIDA -- NÃO ABRIR NO RASTREADOR ***

Título: [Segurança] Token interno gerado em silêncio quando não configurado
Labels: security, severity:baixa, configuracao

Problema

`obter_ou_criar_token_interno` tenta, nesta ordem: variável de ambiente, arquivo
de segredo, arquivo em `instance/interno_token.txt`. Não encontrando nenhum,
gera um `secrets.token_hex(32)` e tenta persistir. Se a escrita falhar – e ela
falha, porque o contêiner tem `read_only: true` – o token vale só para aquele
processo.

Diferente de `obter_chave_secreta`, no mesmo arquivo, este caminho não tem a
trava de `_ambiente_permite gerar_chave`.

Por que importa

O desfecho é seguro: ICT e coletor geram tokens diferentes e a API interna passa
a recusar tudo com 403. Falha fechada, não aberta – por isso a severidade baixa.

O problema é o MODO de falhar. Uma configuração ausente vira uma
indisponibilidade sem causa aparente: a aba Operação para de funcionar por
inteiro e o log só mostra 403 na chamada interna, sem nada que aponte para a
variável que falta. O contraste está a poucas linhas de distância – a chave de
sessão recusa-se a ser gerada fora de desenvolvimento, e a mensagem diz o que
fazer.

Evidência

`app/auth.py:288-296`
```python
    novo_token = secrets.token_hex(32)

```

```

    try:
        caminho.parent.mkdir(parents=True, exist_ok=True)
        caminho.write_text(novo_token, encoding="utf-8")
        with contextlib.suppress(OSError):
            os.chmod(caminho, 0o600)
    except OSError:
        pass
    return novo_token
...

0 padrão correto, no mesmo módulo — `app/auth.py:210-222`:
```python
if not _ambiente_permite gerar_chave():
 raise RuntimeError(
 "Chave de sessao ausente. Em producao ela e obrigatoria: defina "
 "CONFORTO_SECRET_KEY_FILE apontando para /run/secrets/secret_key ..."
)
...

Impacto

Baixo, e não é escalonamento de privilégio. O custo é de diagnóstico, numa
falha que se manifesta longe da causa.

Condição: só ocorre se `CONFORTO_INTERNO_TOKEN_FILE` e
`CONFORTO_INTERNO_TOKEN` estiverem ambos ausentes. O `compose.yaml` define a
primeira para os dois serviços, então não acontece na implantação atual.

Sugestão de correção

Aplicar a mesma trava que a chave de sessão já usa:

```python
if not _ambiente_permite gerar_chave():
    raise RuntimeError(
        "Token interno ausente. Em producao ele e obrigatorio e precisa ser "
        "IGUAL nos dois processos: defina CONFORTO_INTERNO_TOKEN_FILE "
        "apontando para /run/secrets/internal_token (o Compose ja monta "
        "esse segredo) ou CONFORTO_INTERNO_TOKEN. Gerar um token sozinho "
        "faz ICT e coletor discordarem, e a API interna recusa tudo com 403."
    )
...

## Critérios de aceite

- [ ] Fora de desenvolvimento/teste, a ausência das duas variáveis levanta `RuntimeError` no
  boot
- [ ] A mensagem nomeia as duas variáveis e explica a consequência (403 na API interna)
- [ ] Em desenvolvimento (`CONFORTO_DEVELOPMENT=1`), a geração e a persistência em `instance/`
  continuam funcionando
- [ ] Teste cobrindo os dois casos
- [ ] `docs/RUNBOOK.md` registra o sintoma "aba Operação com 403" e aponta para esta
  configuração
--- FIM ISSUE 5 ---

```